

Ace of Spades-Style Clone

Custom Engine Feature Specification and Build Scope

Purpose	Defines the systems, features, and acceptance criteria for a solo-built Ace of Spades-style multiplayer voxel FPS using a custom C++ engine.
Target outcome	A playable prototype first, followed by a solid vertical slice with networking, destructible voxel terrain, building, and team-based combat.
Design principle	Build only what the game needs. Every subsystem must be testable in a sandbox and then proven inside a thin game layer.
Use of this document	Master checklist, scope control sheet, and milestone reference while building the engine and game together.

1. Project framing

This project is not just a game. It is a custom engine plus a voxel multiplayer FPS built on top of it. To keep scope under control, features are grouped by subsystem and labeled as Foundation, Prototype, Vertical Slice, or Polish.

A feature should only be promoted from paper to implementation when it has all three: a clearly defined API, a sandbox test path, and an in-game acceptance condition.

2. Scope bands

Band	Goal	Ship bar	Meaning
FND	Foundation	Engine usable	Subsystem exists, runs in sandbox, and supports basic debugging.
PRT	Prototype	Core loop works	Player can move, shoot, dig, and place blocks in a functional test map.
VSL	Vertical Slice	Real match flow	Networking, teams, respawn, objective rules, and acceptable performance are in place.
POL	Polish	Quality uplift	Optimization, UX, audio, menus, effects, and content iteration improve the experience.

3. Foundation systems

ID	Feature	Band	Acceptance	Notes / exact scope
ENG-01	Project structure	FND	Engine, sandbox, and game targets build separately.	Separate libraries/executables; debug and release configs; dependency setup.
ENG-02	Logger and assertions	FND	Console/file logging works; failed conditions halt cleanly in debug.	Severity levels, timestamps, categories, assert macros, startup log.
ENG-03	Window abstraction	FND	Create, resize, and close a window from one engine-facing API.	Single backend first; input events and drawable size exposed.
ENG-04	Input system	FND	Keyboard and mouse state can be queried every frame.	Pressed, held, released, relative mouse motion, lock/unlock cursor.
ENG-05	Time and main loop	FND	Stable frame delta and fixed-step update path exist.	Variable render loop with optional fixed simulation tick.
ENG-06	File and config layer	FND	Engine reads config, assets, and save files from known paths.	Path helpers, ini/json parsing, simple hot-reload hooks later.
ENG-07	Debug UI / overlay	FND	FPS, frame time, chunk info, and toggles visible at runtime.	Use immediate-mode UI or equivalent lightweight overlay.

4. Rendering

ID	Feature	Band	Acceptance	Notes / exact scope
REN-01	Graphics context init	FND	Clear screen and draw a test triangle/cube.	Set up device/context and basic error reporting.
REN-02	Buffer abstractions	FND	Vertex/index/uniform buffers can be created and updated.	Enough to draw chunk meshes and debug primitives.
REN-03	Shader system	FND	Compile, bind, and report shader errors.	Uniform setting, file loading, basic permutation handling later.
REN-04	Camera + transforms	FND	World, view, and projection matrices drive rendering correctly.	Perspective camera, free-fly debug camera, frustum parameters.
REN-05	Mesh submission path	PRT	Sandbox submits multiple meshes per	Static mesh draw path first; dynamic update support second.

ID	Feature	Band	Acceptance	Notes / exact scope
			frame without breakage.	
REN-06	Chunk mesh renderer	PRT	Voxel chunks render correctly with hidden faces removed.	Primary renderer requirement for the project.
REN-07	Debug draw	PRT	Lines, boxes, rays, and chunk bounds can be visualized.	Used for collision, raycasts, and network debugging.
REN-08	Culling	VSL	Off-camera chunks are not submitted.	Basic frustum culling; distance culling optional early.
REN-09	Lighting and materials	VSL	World is readable and consistent under simple lighting.	Keep simple: directional light, vertex colors, texture atlas if needed.

5. Voxel world and terrain editing

ID	Feature	Band	Acceptance	Notes / exact scope
VOX-01	Voxel data model	FND	Blocks can be stored, queried, and modified by coordinates.	Block ID, solid/empty state, optional color/material payload.
VOX-02	Chunk layout	FND	World splits into fixed-size chunks with stable indexing.	Chunk size chosen once and documented; neighbor lookup supported.
VOX-03	Chunk storage and lifetime	FND	Chunks can be allocated, destroyed, loaded, and saved.	Required before large worlds or networking.
VOX-04	Face culling mesher	PRT	Only visible faces generate geometry.	Dumb mesher first is acceptable; optimize later.
VOX-05	Dirty chunk rebuild queue	PRT	Editing a block rebuilds only affected chunks.	Must include border update logic for neighbor chunks.
VOX-06	Block placement	PRT	Player can place valid blocks instantly.	Target block, adjacency rule, inventory restrictions later.
VOX-07	Block removal / digging	PRT	Player can remove target blocks instantly.	Immediate feedback and correct mesh refresh required.
VOX-08	Raycast selection	PRT	Player can target the correct block face.	Needed for both digging and placement.
VOX-09	Collision mesh generation	PRT	Player collides with edited terrain correctly.	Can start with per-block broad approach, then optimize.

ID	Feature	Band	Acceptance	Notes / exact scope
VOX-10	Chunk serialization	VSL	World/chunk data persists across sessions.	Versioned format strongly recommended.
VOX-11	Optimized meshing	VSL	Chunk triangle count and rebuild time are acceptable.	Greedy meshing or equivalent if dumb mesher becomes bottleneck.
VOX-12	Streaming / map loading	POL	Maps load quickly and world start-up is reliable.	Needed once match maps become larger than test arenas.

6. Player gameplay loop

ID	Feature	Band	Acceptance	Notes / exact scope
PLY-01	Character controller	PRT	Walk, jump, crouch, and fall feel stable.	No clipping through terrain; clean stair/edge handling.
PLY-02	Camera look	PRT	Mouse look is smooth and independent of frame rate.	Sensitivity setting; lock cursor during play.
PLY-03	Hitscan rifle	PRT	Weapon fires, hits, and damages correctly.	Ammo, reload, basic spread, muzzle flash later.
PLY-04	Shovel / dig tool	PRT	Dedicated terrain-removal tool works reliably.	Can share raycast path with weapon and build tool.
PLY-05	Build tool	PRT	Dedicated terrain-placement tool works reliably.	Visual placement preview strongly recommended.
PLY-06	Health and damage	PRT	Players take damage and can die.	Hit feedback and respawn hooks included.
PLY-07	Inventory / slots	VSL	Weapon, shovel, and block tool can be switched cleanly.	Simple fixed slot layout is enough.
PLY-08	Respawn flow	VSL	Death leads to respawn at valid team spawn.	Respawn timer and spawn protection optional.
PLY-09	Movement tuning	VSL	Movement and gun feel are intentionally tuned.	Acceleration, air control, recoil, and feedback pass.

7. Networking and multiplayer authority

ID	Feature	Band	Acceptance	Notes / exact scope
----	---------	------	------------	---------------------

ID	Feature	Band	Acceptance	Notes / exact scope
NET-01	Client/server architecture	FND	Dedicated or listen server can host a match.	Server authoritative over movement, hits, and terrain edits.
NET-02	Connection flow	FND	Client can join and disconnect cleanly.	Version check, timeout, and basic error messages.
NET-03	Player transform replication	PRT	Other players move smoothly on remote clients.	Interpolation/smoothing required.
NET-04	Terrain edit replication	PRT	Placed/removed blocks stay in sync across peers.	Batch edits or delta packets recommended.
NET-05	Combat replication	PRT	Shots, hits, damage, and deaths remain authoritative.	Keep first pass simple even if responsiveness is imperfect.
NET-06	Game state replication	VSL	Teams, score, timer, and spawn state sync correctly.	Match flow cannot depend on local-only state.
NET-07	Join-in-progress state sync	VSL	Late joiner receives current world and match state.	World snapshot + chunk sync path.
NET-08	Lag handling	POL	Game remains playable under nonzero latency.	Prediction and lag compensation only after basic sync works.
NET-09	Bandwidth/perf pass	POL	Terrain and player replication stay within sane limits.	Profile packet sizes, send rates, and spike cases.

8. Match systems and game rules

ID	Feature	Band	Acceptance	Notes / exact scope
GAM-01	Teams	VSL	Players join a valid team and spawn accordingly.	Minimal team UI and spawn ownership.
GAM-02	Spawn points	VSL	Maps define spawn areas that work after terrain edits.	Protected against obviously invalid spawn states.
GAM-03	Scoreboard	VSL	Kills, deaths, teams, and score display correctly.	Simple overlay is enough for first version.
GAM-04	Match timer/state	VSL	Warmup, active match, and end-of-match states exist.	Server controls progression.

ID	Feature	Band	Acceptance	Notes / exact scope
GAM-05	Objective mode	VSL	At least one real win condition is implemented.	Capture, territory control, or ticket system—pick one only.
GAM-06	Map rotation	POL	Server can load the next map without manual rebuild.	Small map list and config support.

9. Tooling and development support

ID	Feature	Band	Acceptance	Notes / exact scope
TOL-01	Sandbox executable	FND	Engine features can be tested without full game rules.	Mandatory development surface for every subsystem.
TOL-02	Debug visualizations	FND	Toggle chunk bounds, collision, raycasts, and net stats.	Used continuously during development.
TOL-03	Map authoring path	PRT	Developer can create or edit test maps quickly.	Can be file-based, editor-less, or in-game tools first.
TOL-04	Profiling hooks	PRT	Frame timing and chunk rebuild timing are visible.	CPU timing first; GPU/network views later.
TOL-05	Crash and error capture	VSL	Meaningful diagnostics exist when something fails.	Logs, assertions, and basic minidump or stack capture if feasible.

10. UX, presentation, and quality passes

ID	Feature	Band	Acceptance	Notes / exact scope
POL-01	Audio layer	PRT	Gunshots, footsteps, impacts, and UI sounds play correctly.	3D positional audio for gameplay-critical cues.
POL-02	HUD	PRT	Crosshair, health, ammo, and block/tool state visible.	Readable and low-overhead.
POL-03	Main menu/server browser	VSL	Player can launch, host, and join from a basic menu.	Keep minimal if server list is deferred.
POL-04	Settings	VSL	Video, audio, and input basics can be adjusted.	Sensitivity and resolution matter early.

ID	Feature	Band	Acceptance	Notes / exact scope
POL-05	Visual feedback	VSL	Hit markers, damage cues, build indicators, and death feedback exist.	Game feel feature, not optional forever.
POL-06	Optimization pass	POL	Average play session maintains target frame rate.	Profile-driven only; no premature engine rewrites.

11. Recommended implementation order

1. Build foundation systems first: logger, window, input, timing, debug overlay, and a sandbox app.
2. Get the renderer drawing chunk meshes before building advanced gameplay.
3. Finish voxel editing and collision before networking. Single-player proof comes first.
4. Implement the complete offline gameplay loop: move, shoot, dig, place, die, respawn.
5. Only then add multiplayer authority and replication, starting with player transforms and terrain edits.
6. Add match rules and presentation after the core loop works in networked play.
7. Treat polish and optimization as final passes driven by measured bottlenecks.

12. Definition of done for the first playable milestone

- A server and at least one client can join the same map.
- Players can move, shoot, dig, place blocks, take damage, die, and respawn.
- Terrain edits update rendering and collision correctly on all connected peers.
- At least one map and one objective rule can be played end to end.
- The game can be debugged using logs, overlays, and chunk/raycast visualization.